Déploiement d’un nœud Ethereum

J’ai déployé Erigon v3.8.1 sur le VPS de l’école, avec Caplin pour le consensus. J’avais commencé avec Geth et Lighthouse, puis changé de client après le conseil du professeur. Les tests et les captures présentés ici viennent du VPS ou de requêtes envoyées depuis mon PC.

État vérifié et limites du rendu

Dernier relevé UTC : 2026-11-01T12:45:19.757947+00:00. Objectif de rendu : le 1er novembre 2026 à 23 h 59, heure de Paris.

Le RPC local expose le bloc 26,316,998. Son horodatage a environ 10.05 heures de retard. Ce retard n’est pas une durée de synchronisation restante.

La référence indépendante https://ethereum-rpc.publicnode.com expose le bloc 26,320,005, soit 3,007 blocs d’écart à ce relevé. Comparaison du hash à notre hauteur : identique. La référence ne remplace pas notre nœud : elle sert seulement à contrôler son rattrapage.

Pour tenir l’échéance, le budget calculé demande en moyenne 1 000 blocs validés en au plus 102 minutes, nouveaux blocs du réseau compris. C’est un seuil à atteindre, pas une prévision de fin ni un débit déjà obtenu.

Synchronisation : pas encore confirmée. Un téléchargement terminé ou eth_syncing=false avec une tête ancienne ne suffit pas.

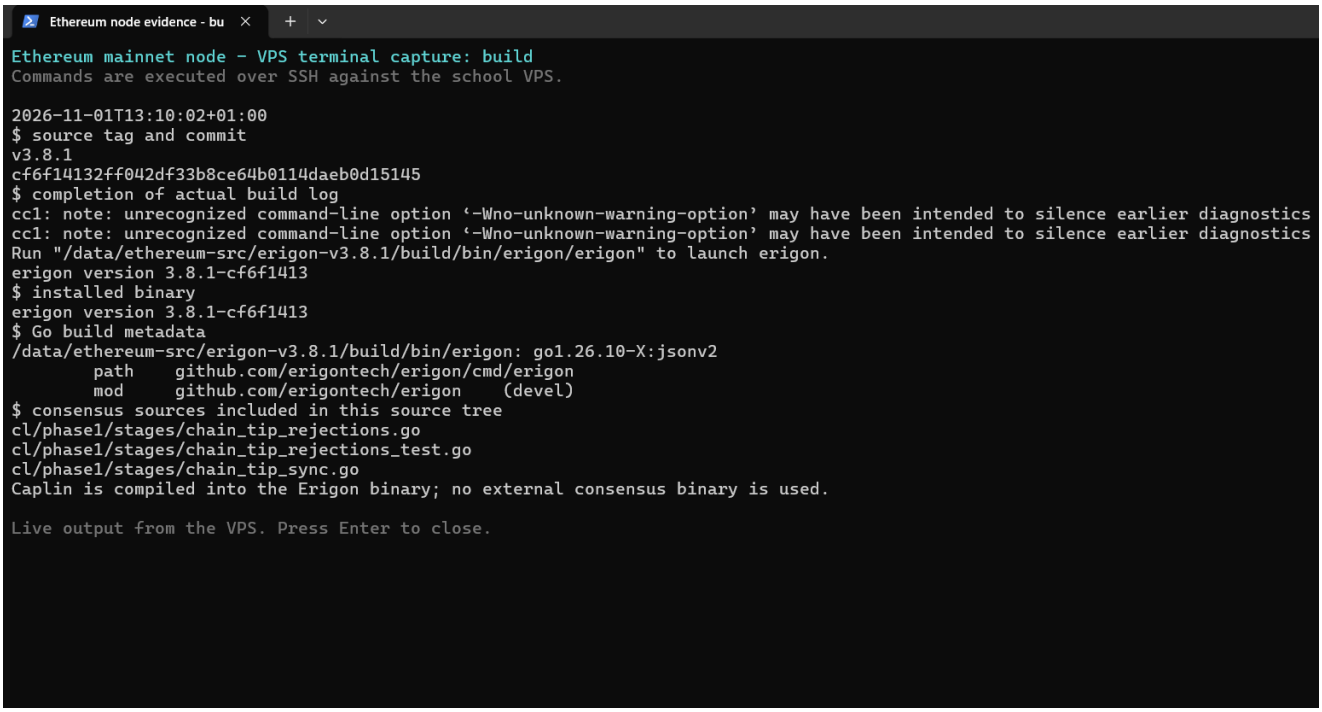
Écart au sujet : la base a été créée en mode minimal, pas avec le mode full d’Erigon. Je ne présente pas ces modes comme équivalents. Changer de mode demande une resynchronisation selon Erigon; je conserve les données existantes au lieu de supprimer la base à l’approche du rendu.

Le démarrage automatique est configuré. Le test réel de reboot et de récupération reste à valider. Le lien GitHub était déjà présent dans la cellule F22 du tableur de rendu, vérifiée le 1er novembre sans modification.

Machine et compilation depuis les sources

La VM utilise Debian 13, 8 vCPU et environ 22 Gio de RAM. Le volume système fait 197 Gio et /data 984 Gio. Le sujet mentionne 2 To; le professeur a indiqué qu’un VPS de 1 To devait suffire. Le volume /data présente une latence mesurée élevée; le drapeau virtuel ROTA ne permet pas de conclure au type du disque physique.

J’ai compilé le tag officiel v3.8.1, commit cf6f14132ff042df33b8ce64b0114daeb0d15145, avec Go 1.26.10 et make erigon. Le Go système 1.24.4 était trop ancien, donc j’ai utilisé directement le toolchain du cache Go. Caplin est intégré : le même build compile le client d’exécution et les sources du consensus. Les avertissements C/MDBX du build n’ont pas empêché la production du binaire. Les commandes de reprise sont dans le README.

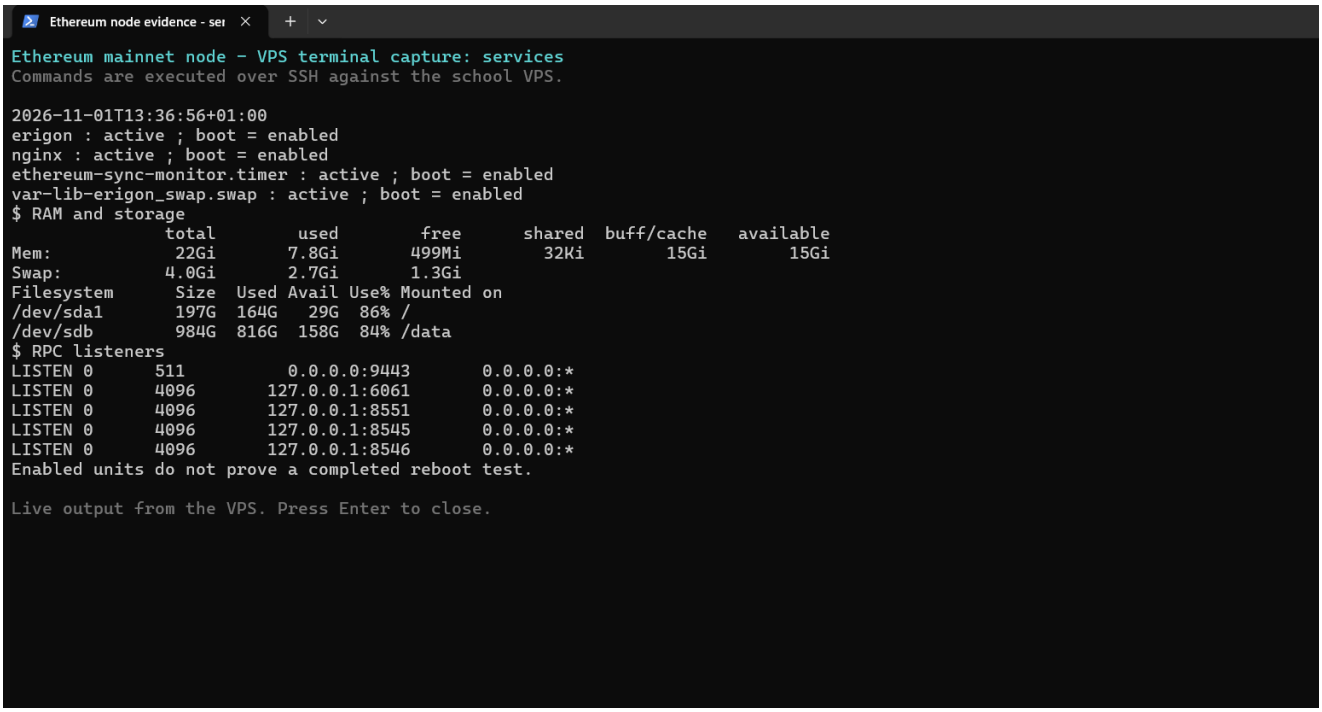


Commit source, fin du journal de compilation et version du binaire réellement installé.

Configuration et services

Le service tourne avec le compte non-root adm-mesifi. Les lots sont réglés à 2 Go, la limite de boucle à 10 000 blocs et le calcul parallèle de commitment reste validé. Erigon et Caplin partagent le processus et le datadir. Les unités Erigon, Nginx, le timer de suivi et le swap de secours sont activés au démarrage. Restart=on-failure relance Erigon en cas d'arrêt anormal; cela ne remplace pas un test de reboot.

Les API HTTP 8545, WebSocket 8546, Engine 8551 et les métriques 6061 restent sur localhost. Les namespaces activés sont eth, net et web3. L'API Beacon reste désactivée pour éviter sa consommation mémoire supplémentaire; les logs et métriques Caplin donnent le suivi du consensus sans l'exposer sur Internet.

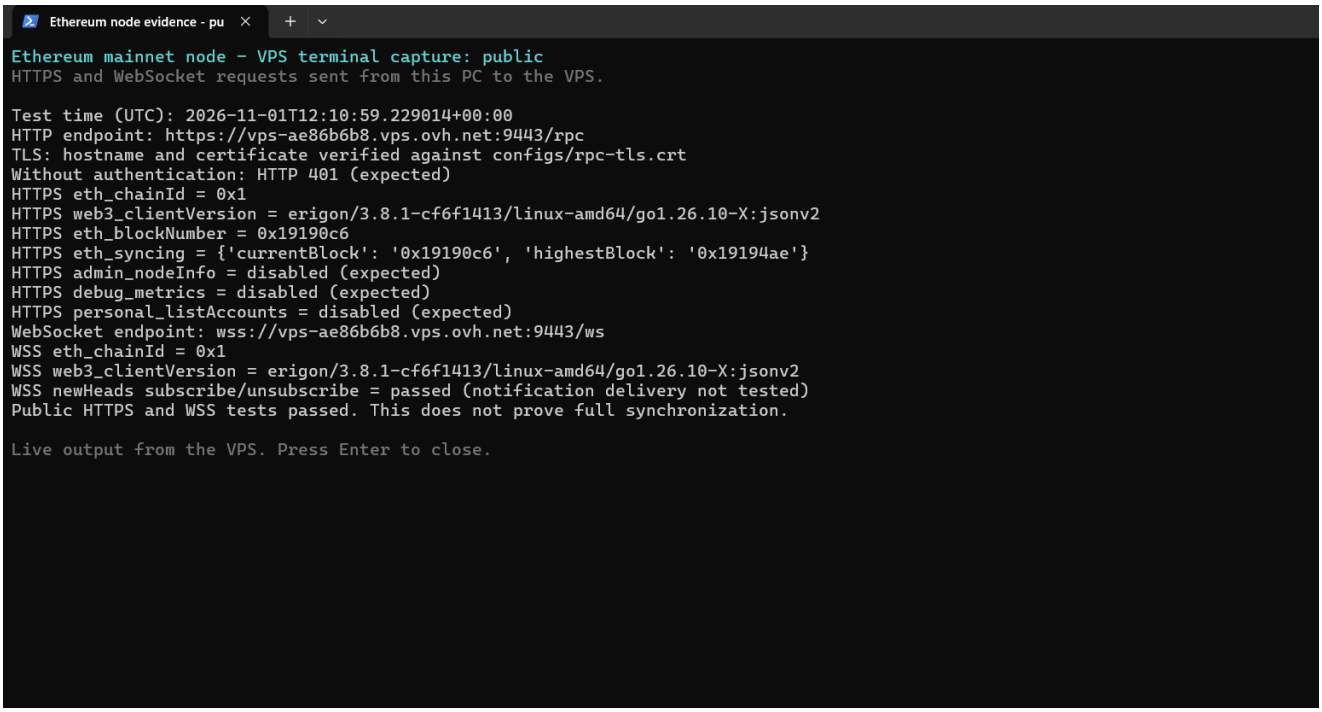


Services actifs et activés au démarrage, ressources et sockets du VPS. Pas une preuve de reboot.

RPC publics et sécurité

Nginx expose HTTPS sur https://vps-ae86b6b8.vps.ovh.net:9443/rpc et WSS sur le même hôte, chemin /ws. L'accès demande une authentification HTTP Basic sur TLS. Le certificat est auto-signé : le script de test lui fait explicitement confiance et vérifie le nom du serveur. La clé privée et les mots de passe ne sont pas dans Git. Les limites du proxy sont 5 requêtes par seconde, 5 connexions par IP et 64 Ko par requête.

Depuis le PC, j'ai vérifié le refus sans identifiants (401), eth_chainId=0x1 et web3_clientVersion sur les deux transports. Les méthodes admin_nodeInfo, debug_metrics et personal_listAccounts sont refusées. L'abonnement et le désabonnement newHeads passent en WebSocket; ce test ne prouve pas la réception d'une notification. Les réponses brutes sont conservées dans evidence/current-public-rpc.log.



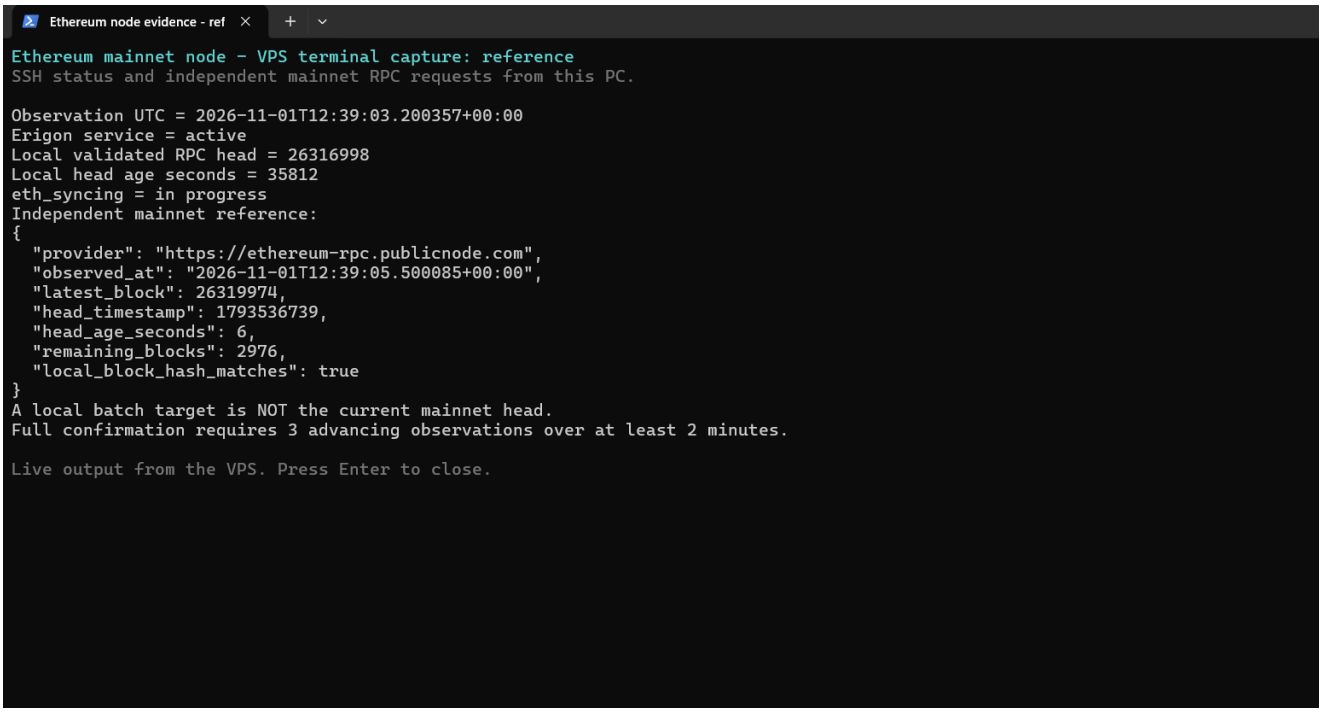
Tests réels depuis le PC vers les endpoints publics, sans afficher les identifiants.

Synchronisation et mesure du rattrapage

Le timer enregistre les mesures toutes les deux minutes sur le VPS. Il relève la tête RPC, les logs d'exécution et de commitment, les événements OOM, la RAM, le swap, les débits et latences disque. Ce suivi fonctionne même quand le PC est éteint. Le PC ajoute la comparaison avec un endpoint mainnet indépendant, accessible depuis le PC mais refusé depuis ce VPS.

La confirmation finale demande trois observations sur au moins deux minutes : service actif, eth_syncing=false, tête de moins de deux minutes, hash identique à la référence et écart de quatre blocs maximum. Le suivi du VPS seul ne constitue pas cette confirmation indépendante.

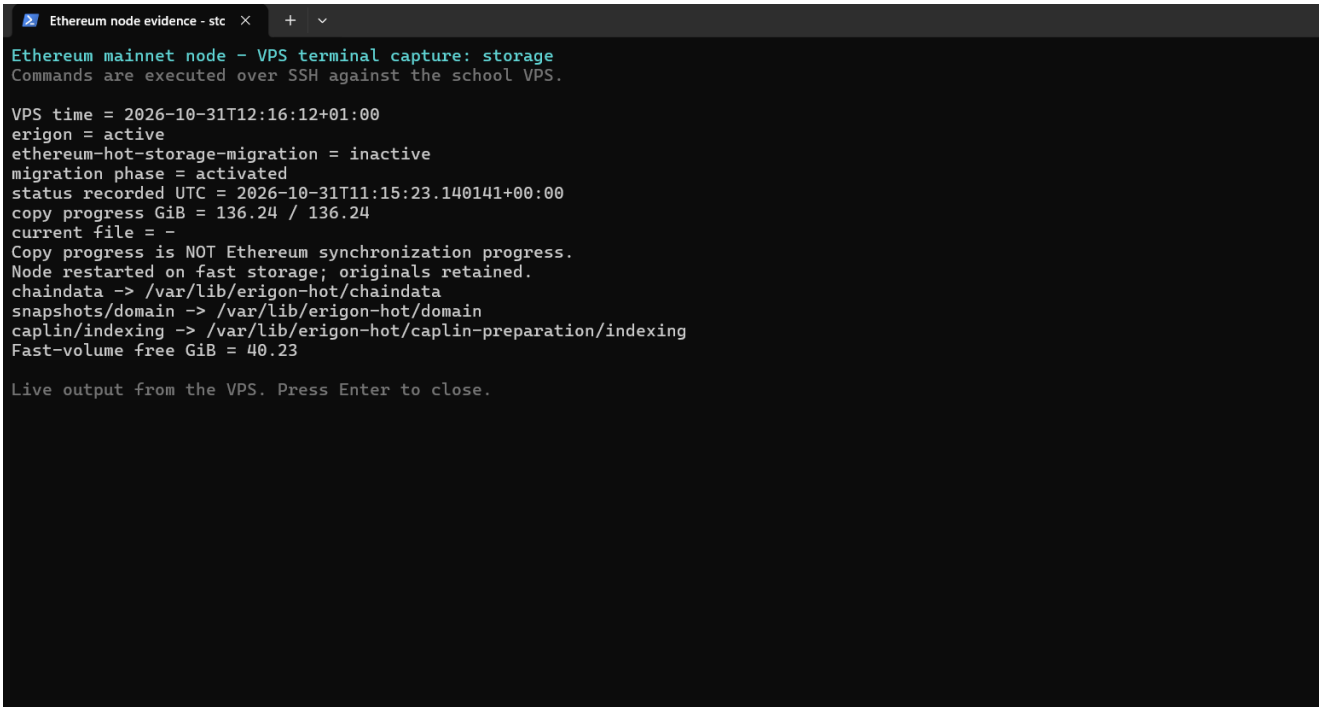
Pas encore d'ETA robuste après la relance mémoire : le calcul attend au moins deux cycles complets de publication du même processus. Une fenêtre courte autour d'un saut de 1 000 blocs peut donner une fausse impression de vitesse. Je ne donne pas de probabilité chiffrée de finir avant l'échéance avec ces seules mesures.



État réel de synchronisation à l'heure affichée dans le terminal. Les captures ne sont pas des estimations.

Incidents et améliorations réalisées

Les lectures aléatoires de contrôle du 31 octobre donnaient environ 0,22 ms sur le volume système contre 31,82 ms sur /data. J'ai donc déplacé chaindata, les fichiers d'état qui tenaient sur le volume rapide et les bases récentes de Caplin. Les copies ont été vérifiées par SHA-256 et les originaux gardés. Les gros fichiers de commitment restent en partie sur /data. Une fusion de snapshots en arrière-plan partage encore ses accès disque avec l'exécution; je n'interromps pas ce travail sans gain démontré.

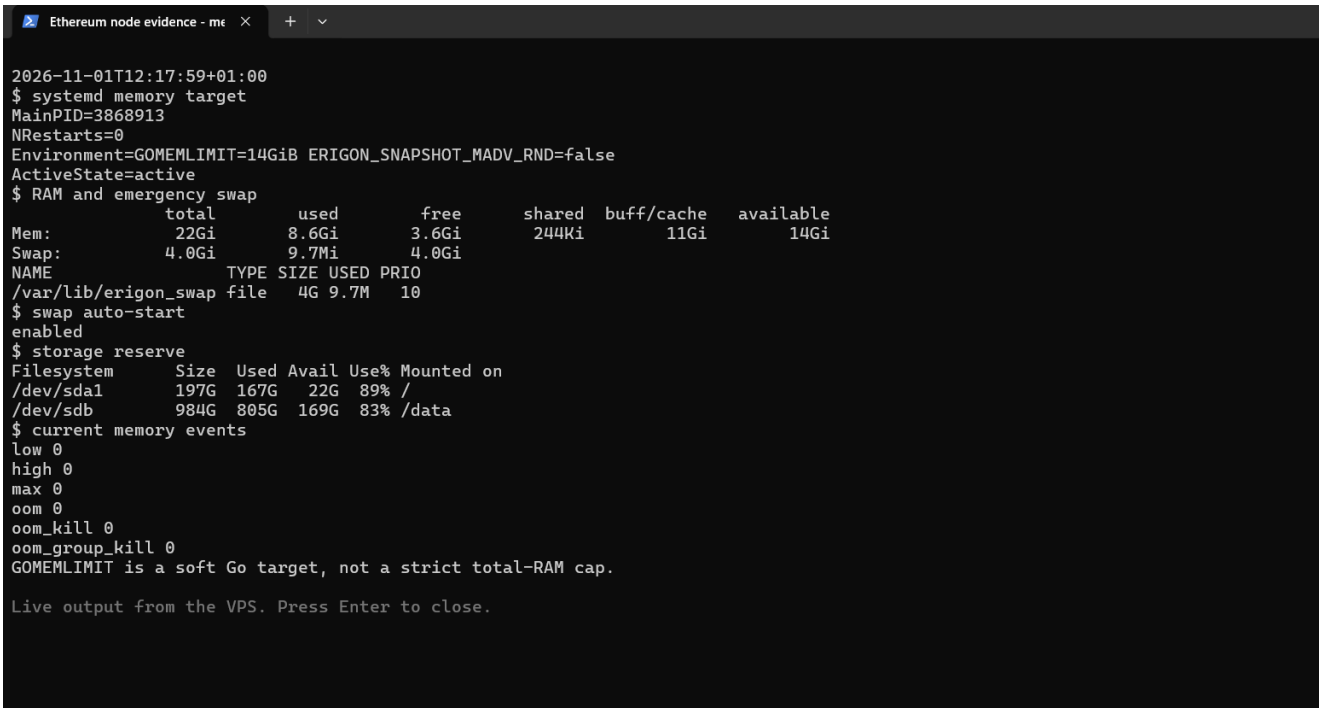


Preuve historique de la bascule du 31 octobre. L'espace libre affiché est celui de cette capture.

Un compteur de blocs est resté fixe pendant un calcul de 383 500 clés qui a duré environ 12 minutes. Le journal détaillé a ensuite montré la reprise : ce compteur fixe n'était pas la preuve d'un deadlock. L'erreur de racine du 31

octobre à 02:25 est apparue après une demande d'arrêt; je ne peux pas l'attribuer avec certitude au mode parallèle.

Le noyau a tué Erigon par manque de RAM le 31 octobre à 22:32 puis le 1er novembre à 07:17. Le service avait redémarré automatiquement. J'ai installé GOMEMLIMIT=14GiB et un swap de secours de 4 Gio sur le volume rapide. Cette cible Go est souple et ne couvre pas toute la mémoire du processus. Le swap n'est pas une accélération et peut ralentir s'il est très utilisé. Une relance propre a appliqué la cible à 12:16. Le verrouillage de 3 Gio d'index en RAM a aussi été retiré après leur déplacement sur le disque rapide.



Configuration mémoire réellement appliquée et compteurs OOM du nouveau processus.

Les 38 tests exécutés sur Linux le 1er novembre passent : copie vérifiée et retour arrière, restauration des réglages, sécurité du swap, diagnostic d'activité, prévisions, qualification de synchronisation et garde-fous des opérations préparées. Le contrôle ciblé de Git n'a trouvé ni les mots de passe SSH/RPC connus ni de clé privée PEM; ce n'est pas un audit exhaustif de tous les secrets possibles.

Le 1er novembre après 13 h, la fusion des fichiers de commitment a fait baisser la réserve du disque système. J'ai réduit sa réserve ext4 de 8 à 4 Gio, conservant environ 4 Gio pour les services privilégiés, et déplacé les caches de compilation vers /data avec vérification des copies. Les fichiers existants de domain restent accessibles par liens; les nouveaux snapshots et temporaires sont désormais créés dans /data/erigon-domain-active. Un arrêt propre puis une relance ont appliqué ce placement vers 13:20. Ce placement réduit le risque de saturation du volume système, sans prouver un gain immédiat de vitesse. Les fichiers d'état utiles n'ont pas été supprimés.

Seul le déplacement du cache go-build, environ 3 Gio, a été terminé et vérifié. La copie secondaire des modules a été interrompue quand la marge est redevenue suffisante : le cache des modules original est intact et sa copie partielle sur /data n'est pas utilisée.

Rendu et références

Dépôt : github.com/jordiparis165/ethereum-node-workshop. Le README décrit les dépendances, la compilation, les unités et les commandes de contrôle. evidence/submission-audit.json distingue les points vérifiés et les manques. Les identifiants RPC doivent être transmis séparément au correcteur. Aucune note sur 20 n'est déduite automatiquement de ces tests.

Documentation utilisée : Caplin intégré, modes de pruning, performance, stockage et limite mémoire Go.